



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

APAI LAB 01: PULP Embedded Programming

Călin Diaconu

Contact: calin.diaconu2@unibo.it

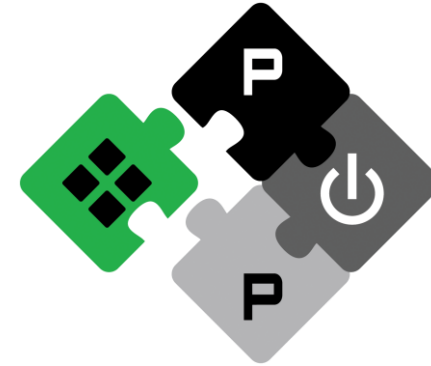


ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

01. Lab Information

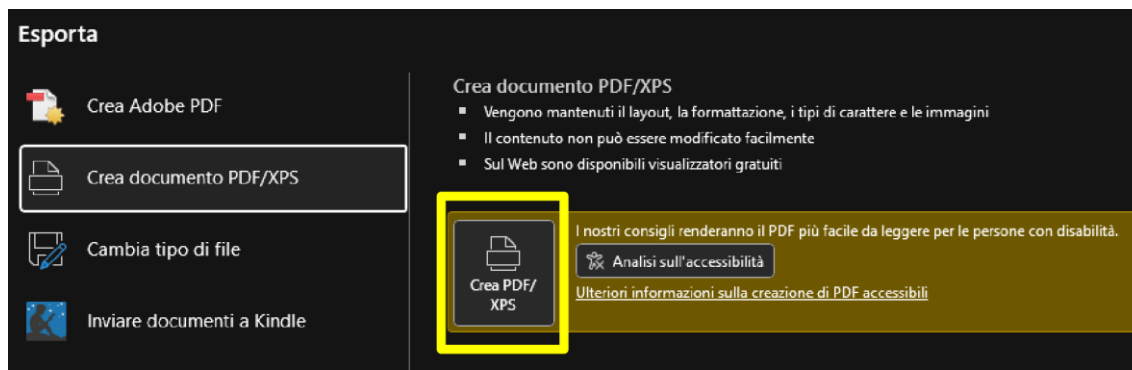
01. Lab Information – Objective of the Class

- **Intro:** PULP platform and the PULP-SDK
- **Tasks:** some basics of C programming on PULP:
 - Hello world example
 - Vector sum
 - Matrix-vector multiplication
 - Measuring execution performance
- **Programming language:** C
- **Lab duration:** 3h
- **AI policy:** we will not prevent you from using AI in solving the assignments, but the lab is meant to be a hands-on learning experience. To keep it useful, we have included an .md file (also used in a [Stanford class](#)), that should be automatically read by AI agents, in order to prevent them to give you a direct solution and rather guide you towards finding it yourself.
- The class is meant to be interactive: coding together and on your own!



01. Lab Information – How to Deliver the Assignment

- You will deliver **ONLY THE PDF** assignment, **NO code**
 - Download the **deliverable template** from Virtuale
 - **Fill in the results** required by the assignment
 - **Export to PDF** format (this option is available in most document editors – Google Docs, Office Word, or as a “Print to PDF” option in the Print menu)
 - **Rename the file** to *LAB<number_of_the_lesson>_APAI_<name_surname>.pdf*
 - Use the **Virtuale platform** to **upload the PDF** before the **deadline (Sunday 18th of October, end of day)**
 - For each lab that you solve and submit successfully, you will receive **1 extra point**, up to a **maximum of 4 points**



LAB06 - On-Device Learning

Opened: Friday, 14 November 2025, 3:30 PM

Due: Thursday, 8 January 2026, 11:00 AM

Upload your completed PDF assignment for LAB06 here!!

[Add submission](#)

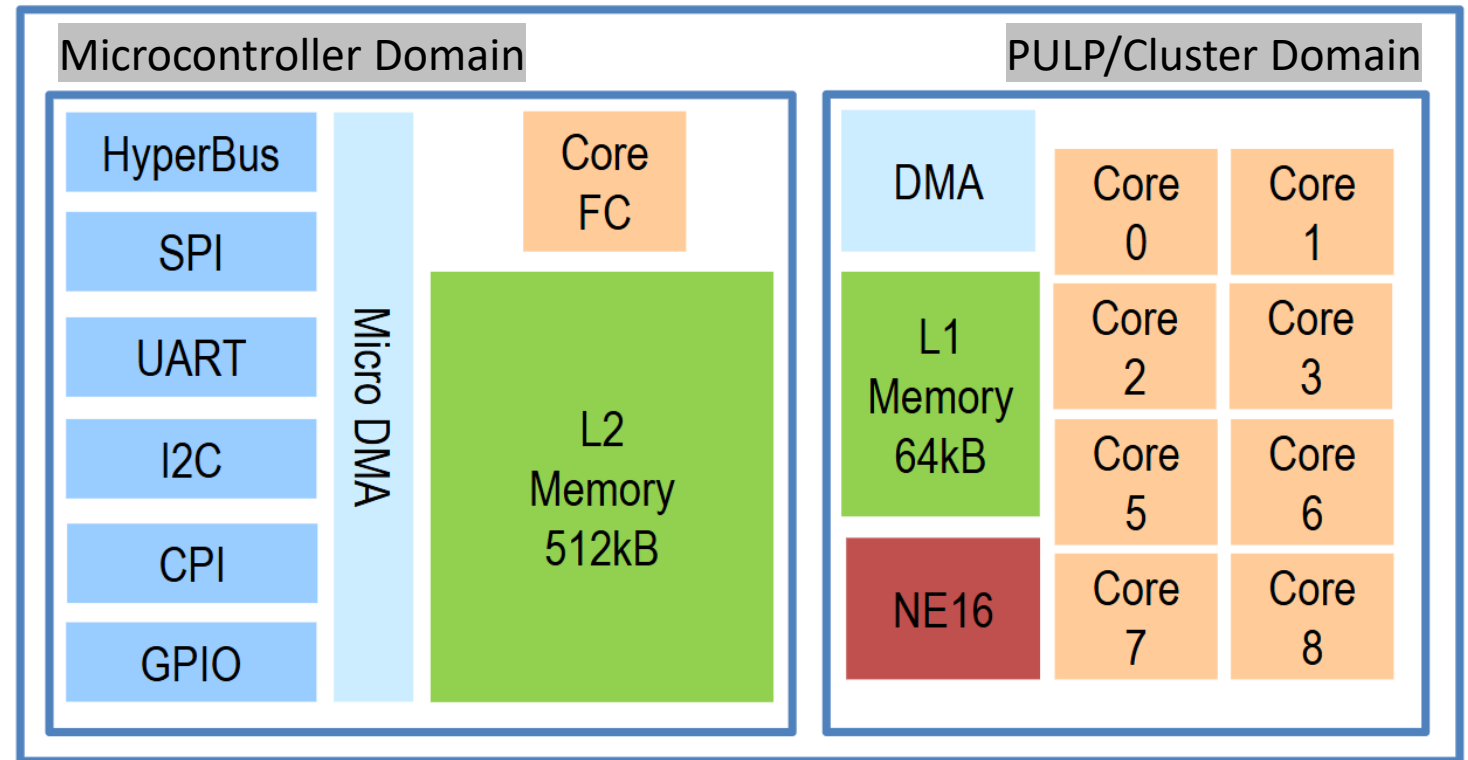




ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

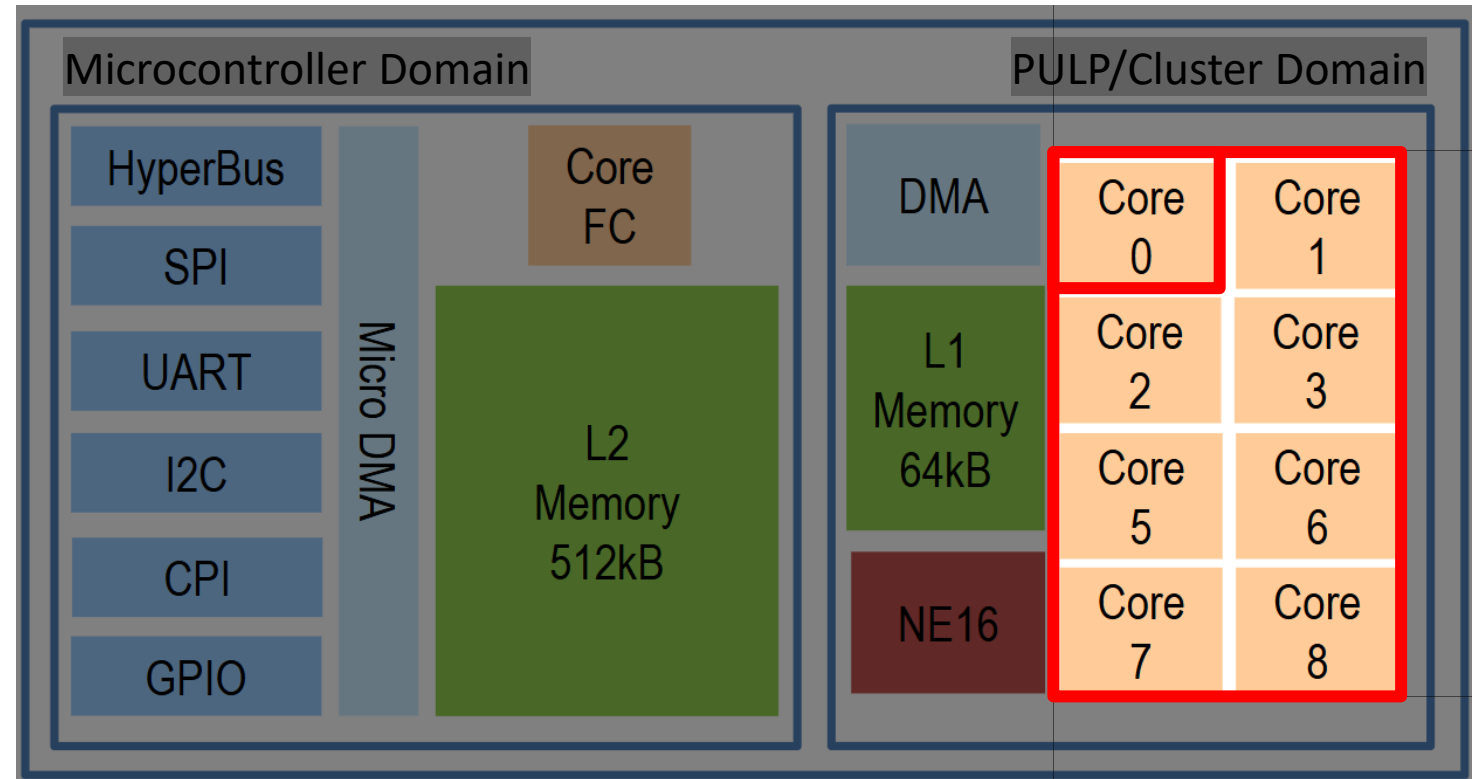
02. Introductory Information

02. Intro Information – The PULP Platform



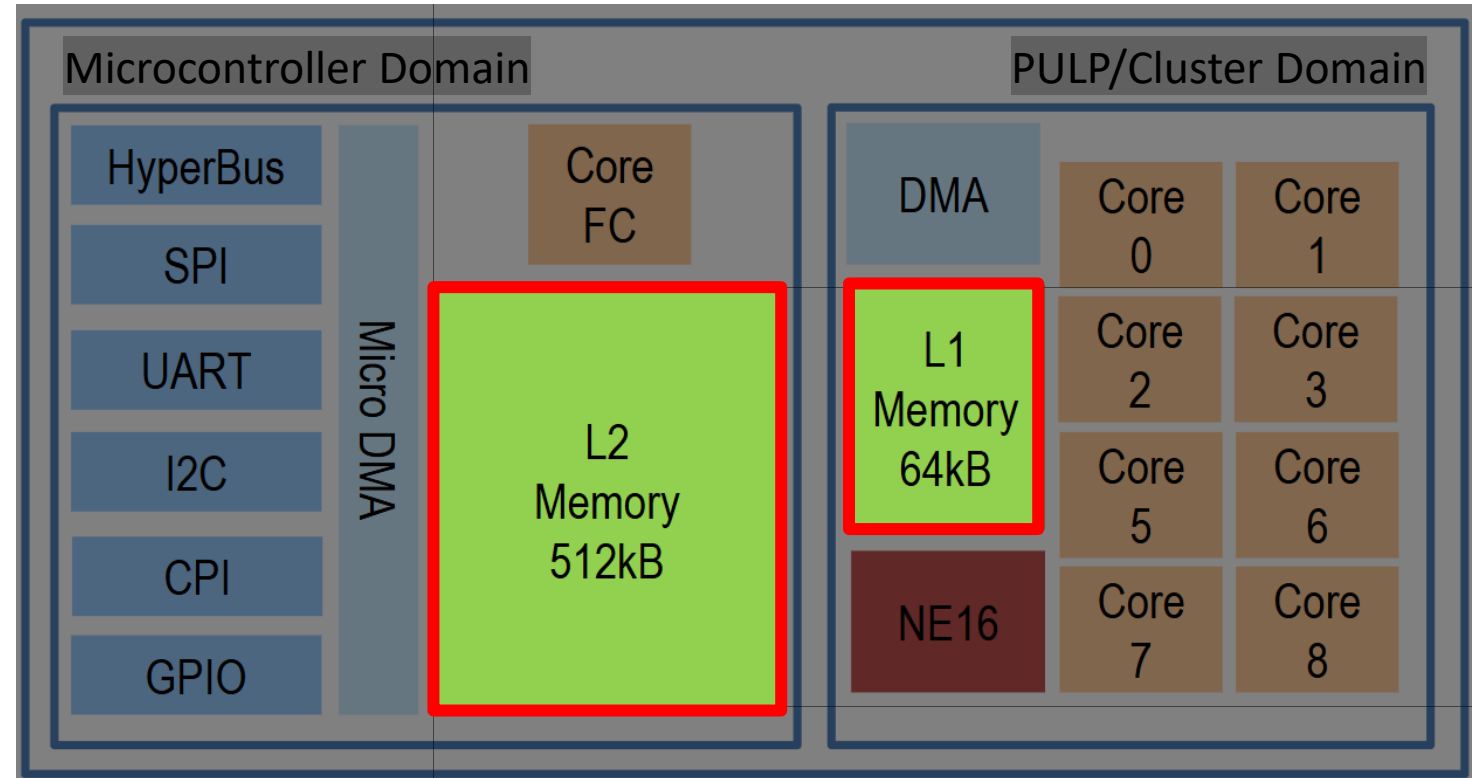
02. Intro Information – The PULP Platform

- **Cores:** 1 + 8 high-performance, with extended RISC-V ISA



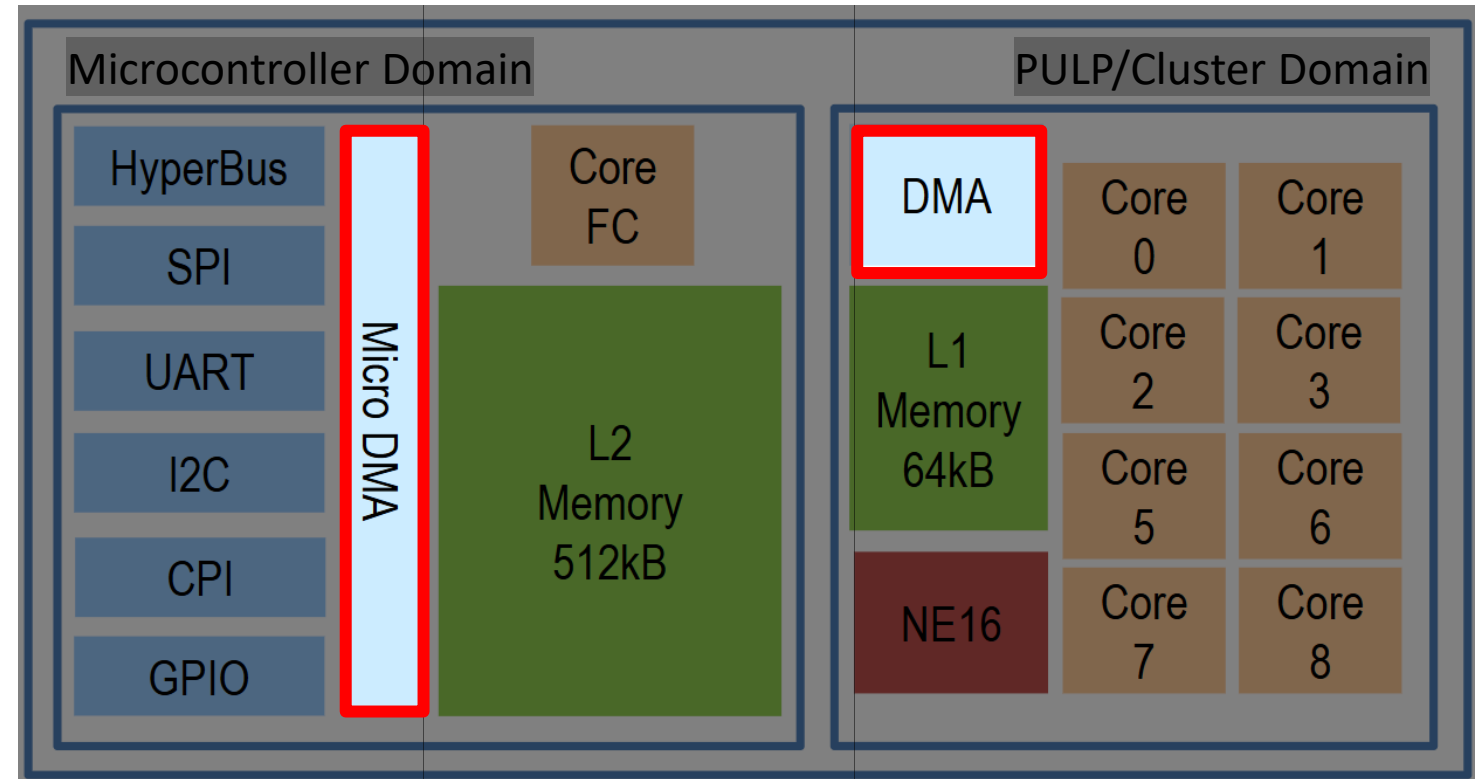
02. Intro Information – The PULP Platform

- **Cores:** 1 + 8 high-performance, with extended RISC-V ISA
- **On-chip Memories**
 - **Level 2** memory for shared with the microcontroller
 - **Level 1** memory shared only by the cores in Cluster



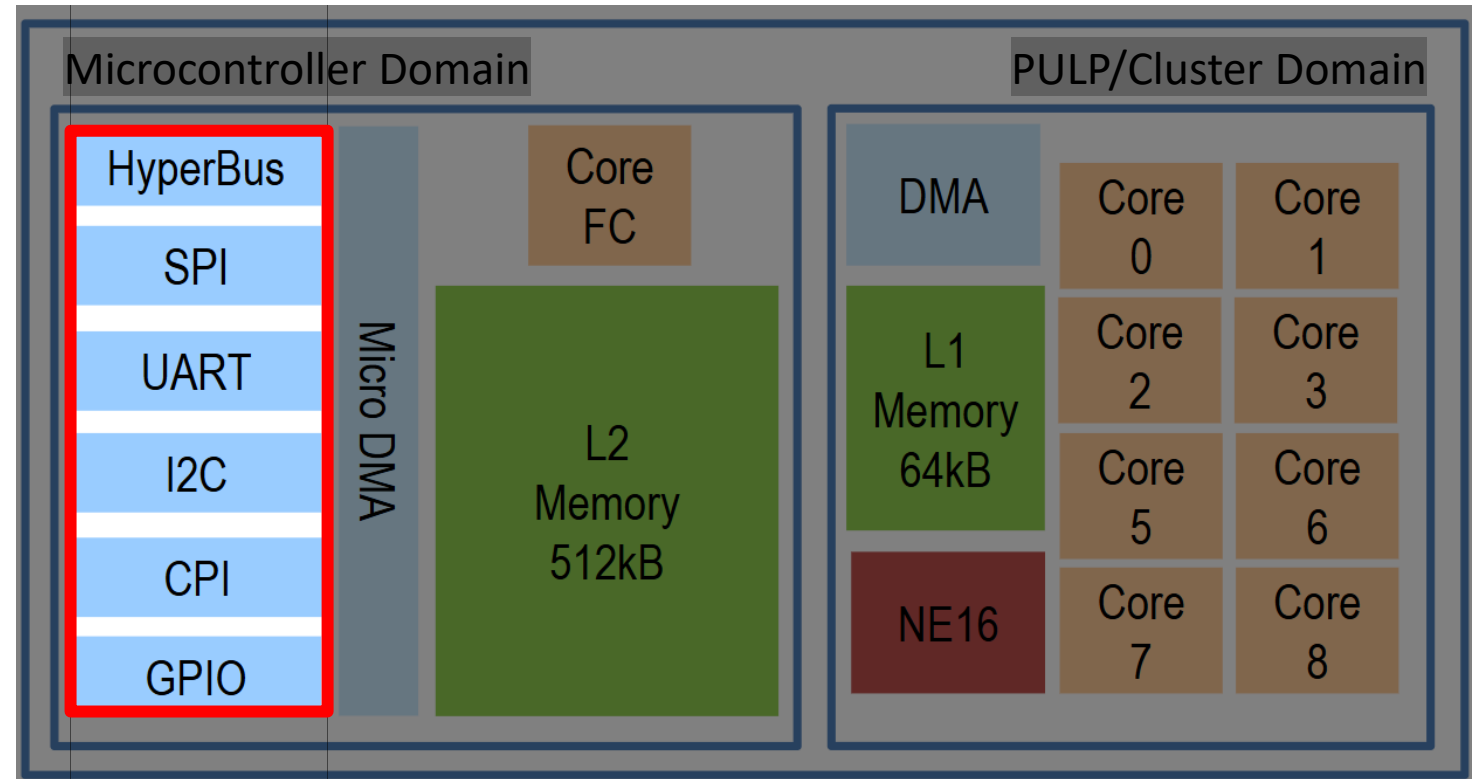
02. Intro Information – The PULP Platform

- **Cores:** 1 + 8 high-performance, with extended RISC-V ISA
- **On-chip Memories**
 - **Level 2** memory for shared with the microcontroller
 - **Level 1** memory shared only by the cores in Cluster
- **cluster-DMA:** multi-channel 1D/2D cluster-DMA that controls the transactions between L1 and L2 Memory
- **micro-DMA:** A smart, lightweight and completely autonomous DMA capable of handling complex I/O operations



02. Intro Information – The PULP Platform

- **Cores:** 1 + 8 high-performance, with extended RISC-V ISA
- **On-chip Memories**
 - **Level 2** memory for shared with the microcontroller
 - **Level 1** memory shared only by the cores in Cluster
- **cluster-DMA:** multi-channel 1D/2D cluster-DMA that controls the transactions between L1 and L2 Memory
- **micro-DMA:** A smart, lightweight and completely autonomous DMA capable of handling complex I/O operations
- **Bus & Peripherals:** HyperBus, I2S, CPI, timers, SPI, GPIOs, etc...



02. Intro Information – Build Automations with Makefiles

- **Building automations** is the process of automating the main steps required to **create a software program**, including **compiling, assembling, linking and testing**
- **Make** is one of the most widespread utilities for this purpose
- **Configuration files** are called **Makefiles**

make command example

```
$ make clean all run
```

Clean previous build processes
(*BUILD/* directory)

Compile the program and produce the **binary code**

Run the **binary code** on the target platform

Makefile example

```
APP = test

# This is a comment
APP_SRCS += test.c
APP_CFLAGS += -O3 -g
APP_LDFLAGS +=

include $(RULES_DIR)/pmsis_rules.mk
```

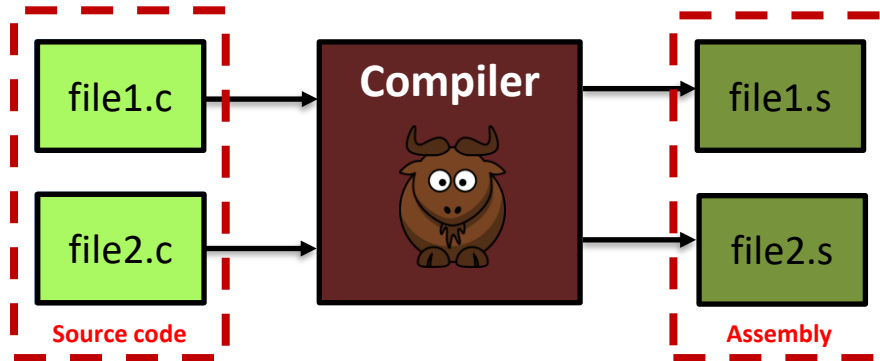


02. Intro Information – Compilation Toolchain

- A **compilation toolchain** includes several tools to achieve its final goal:

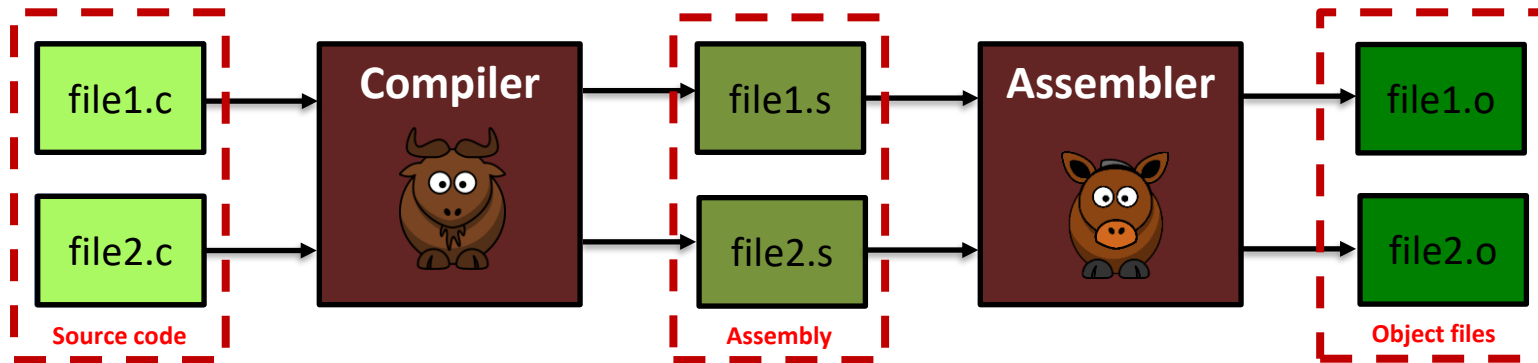
02. Intro Information – Compilation Toolchain

- A **compilation toolchain** includes several tools to achieve its final goal:
 - The **COMPILER** translates high-level **source code** (e.g., C) to a **lower-level representation** (e.g., assembly)



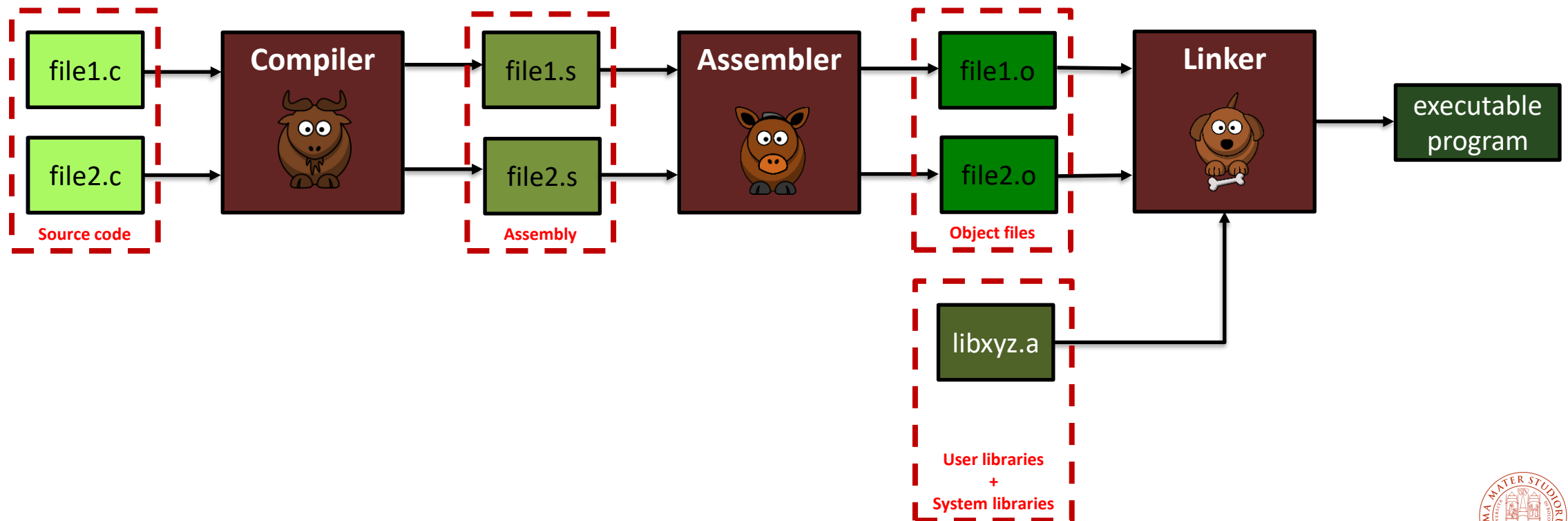
02. Intro Information – Compilation Toolchain

- A **compilation toolchain** includes several tools to achieve its final goal:
 - The **COMPILER** translates high-level **source code** (e.g., C) to a **lower-level representation** (e.g., assembly)
 - The **ASSEMBLER** is a program that translates **assembly language to machine language**



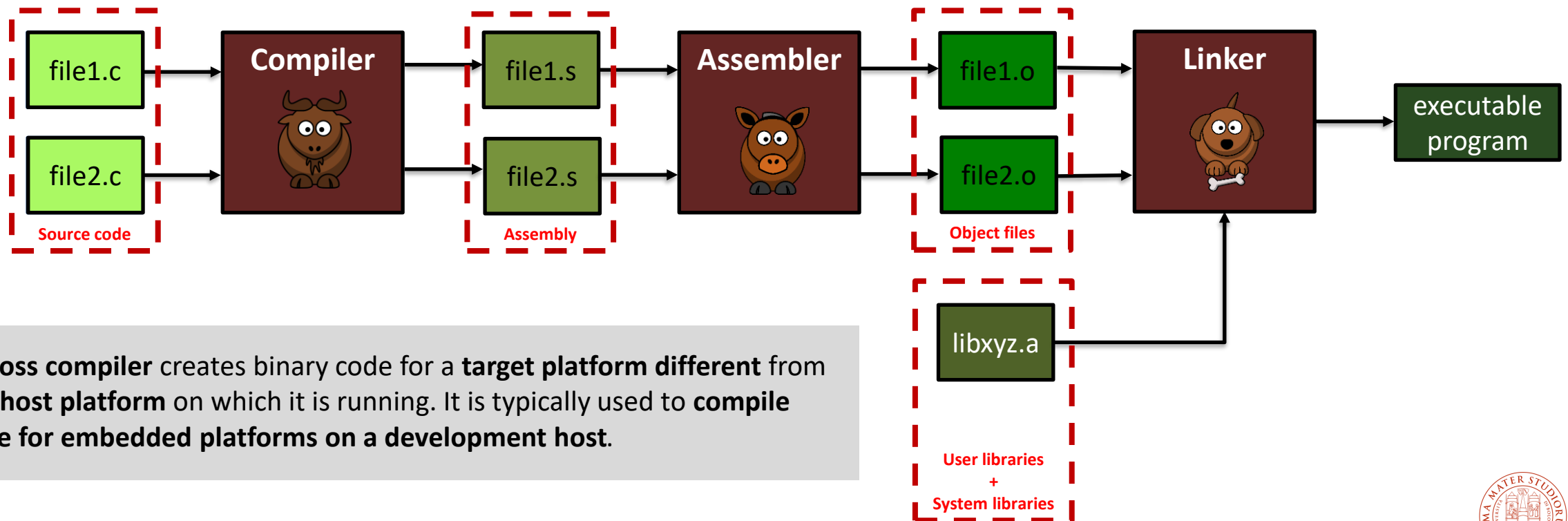
02. Intro Information – Compilation Toolchain

- A **compilation toolchain** includes several tools to achieve its final goal:
 - The **COMPILER** translates high-level **source code** (e.g., C) to a **lower-level representation** (e.g., assembly)
 - The **ASSEMBLER** is a program that translates **assembly language to machine language**
 - The **LINKER** combines **multiple object files into a single executable**



02. Intro Information – Compilation Toolchain

- A **compilation toolchain** includes several tools to achieve its final goal:
 - The **COMPILER** translates high-level **source code** (e.g., C) to a **lower-level representation** (e.g., assembly)
 - The **ASSEMBLER** is a program that translates **assembly language to machine language**
 - The **LINKER** combines **multiple object files into a single executable**



A **cross compiler** creates binary code for a **target platform different** from the **host platform** on which it is running. It is typically used to **compile code for embedded platforms on a development host**.

02. Intro Information – PULP Software Environment Workflow

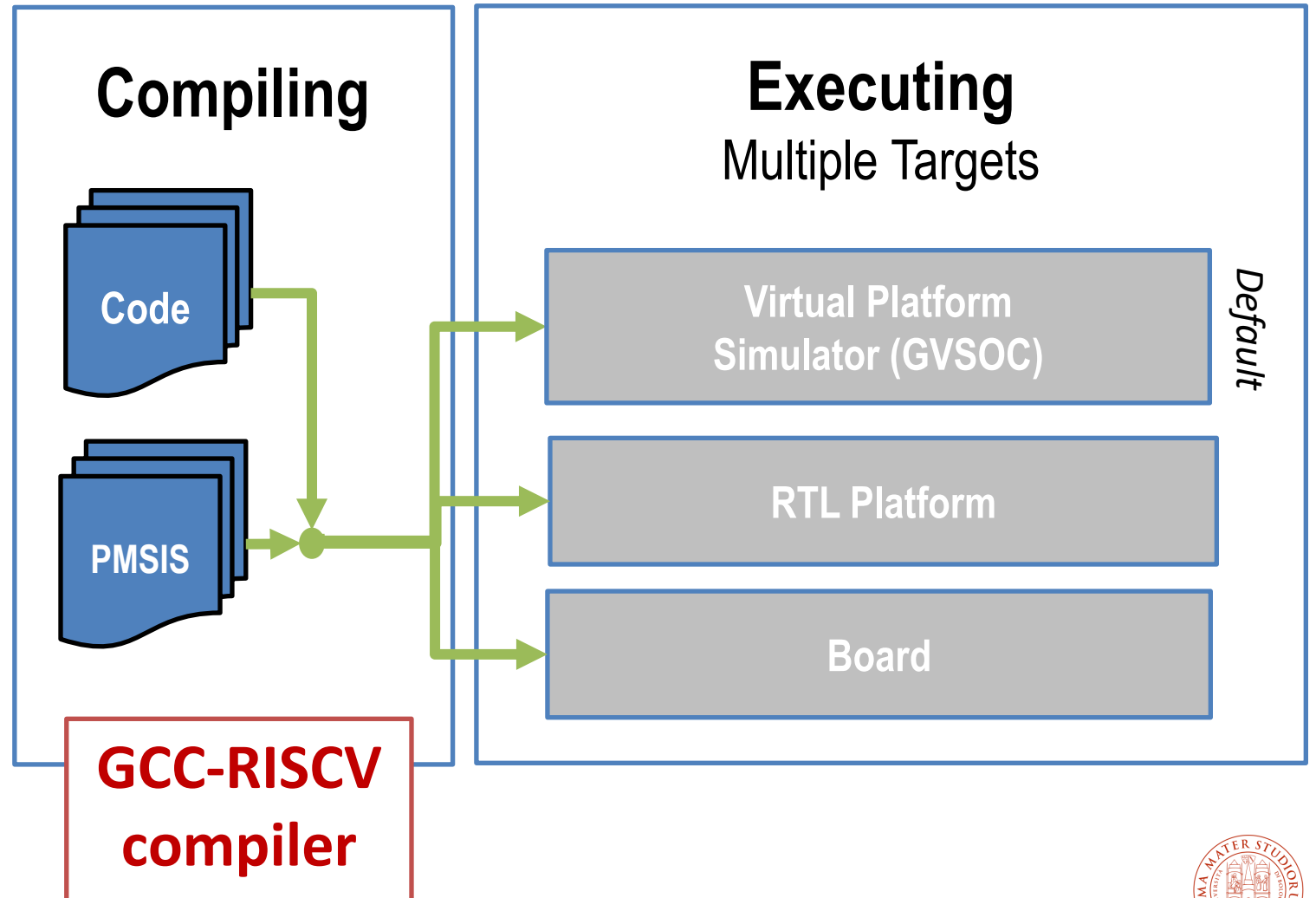
Application code:

main function, application libraries, e.g. CNN inference function code...

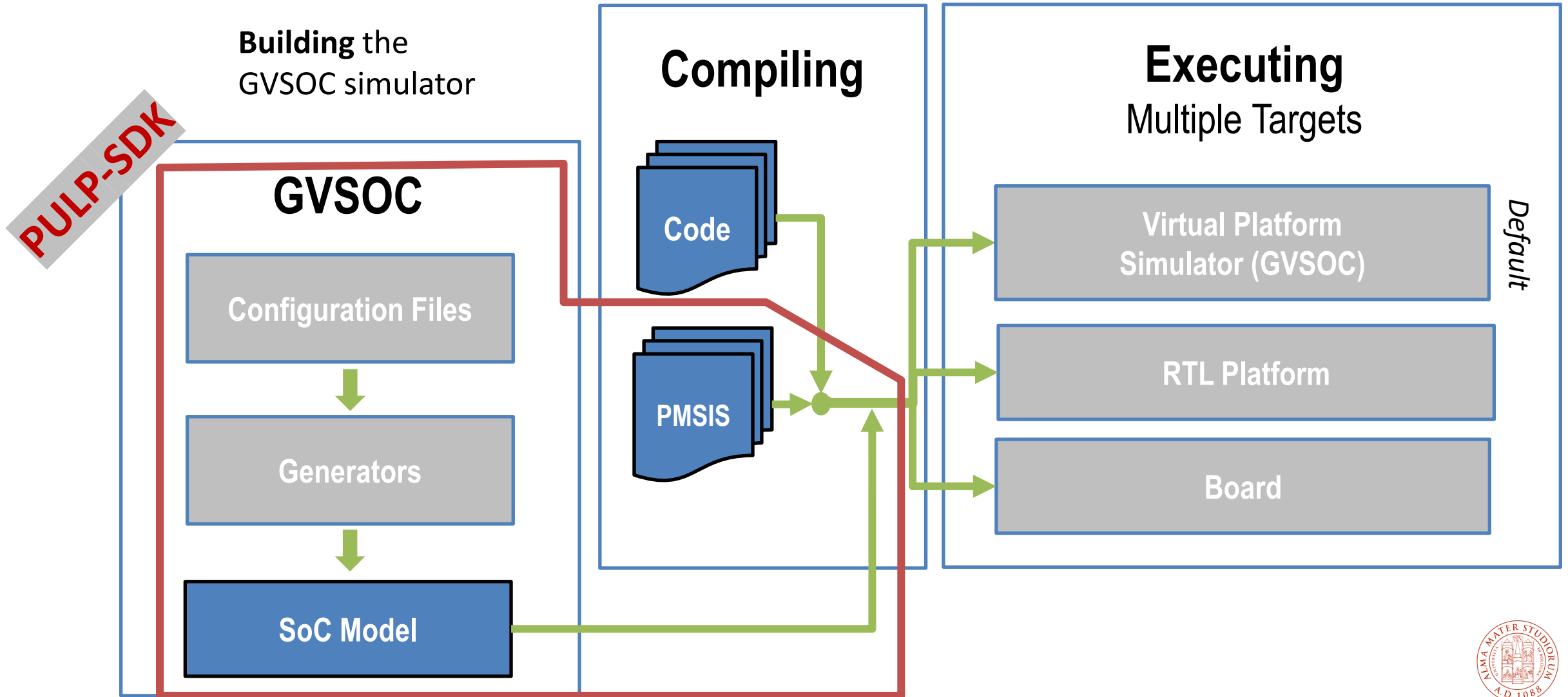
Runtime SW:

Peripheral *Drivers*, API, Board support packages (BSP)

- *PULP Microcontroller Software Interface Standard (PMSIS) layer*



02. Intro Information – PULP Software Environment Workflow



02. Intro Information – PULP SDK

The PULP-SDK (<https://github.com/pulp-platform/pulp-sdk>) includes a **PULP platform simulator (GVSOC)** and the SW libraries.

The */pulp/pulp-sdk* folder contains:

- **rtos/** - runtime code and software stack
- **tools/** - configuration files, python generators, pulp runner and the gvsoc components
- **tests/** - sample code to test the platform's features
- **applications/** - full-application codes

```
pulp-user@pulp-box /pulp/pulp-sdk $ ll
total 60
drwxrwxr-x  9 pulp-user pulp-user 4096 Feb  3  2021 ./
drwxrwxr-x  6 pulp-user pulp-user 4096 Feb  3  2021 ../
drwxrwxr-x  6 pulp-user pulp-user 4096 Feb  3  2021 build/
drwxrwxr-x  2 pulp-user pulp-user 4096 Feb  3  2021 configs/
drwxrwxr-x  8 pulp-user pulp-user 4096 Feb  3  2021 .git/
-rw-rw-r--  1 pulp-user pulp-user   80 Feb  3  2021 .gitignore
drwxrwxr-x  4 pulp-user pulp-user 4096 Feb  3  2021 install/
-rw-rw-r--  1 pulp-user pulp-user 11357 Feb  3  2021 LICENSE
-rw-rw-r--  1 pulp-user pulp-user   877 Feb  3  2021 Makefile
-rw-rw-r--  1 pulp-user pulp-user 2455 Feb  3  2021 README.md
drwxrwxr-x  4 pulp-user pulp-user 4096 Feb  3  2021 rtos/
drwxrwxr-x  2 pulp-user pulp-user 4096 Feb  3  2021 rules/
drwxrwxr-x 10 pulp-user pulp-user 4096 Feb  3  2021 tools/
pulp-user@pulp-box /pulp/pulp-sdk $
```

**DON'T RUN THIS
COMMAND**



You could get the latest update from the open-source community and rebuild the GVSOC simulator

```
$ git pull origin main
$ source configs/pulp-open.sh
$ make build
```



02. Intro Information – GVSoC Features

- **Virtual platform** features:
 - **C++** for fast native **simulation**
 - **Python** for **instantiation + configuration**
 - **Complete set of traces** to see what happens
- **Timing model:**
 - Fully-event based (**cycle-accurate**), instances can generate events at specific time
 - **Includes timing models** for interconnects, DMACs, memories...
 - **Performance counters** for information on the execution
- **Simulation performance:**
 - **Around 1 Million Instructions Per Second (MIPS)**
 - Functionally **aligned and calibrated with HW**
 - Timing accuracy is **within 10-20% of target HW**





ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

03. Prerequisites

03. Prerequisites – Containers & Docker



- **What is a Container?**
 - A lightweight, portable, and self-contained environment that bundles together
 - A consistent runtime environment
 - Your application code
 - All its dependencies (libraries, tools, configurations)
- **What is Docker?**
 - The most used platform for working with containers, which allows the user to:
 - **Build images**, defined in a Dockerfile
 - **Run containers** – isolated environments, created according to the “recipe” defined inside an image
 - **Share images** to other people through the Docker Hub, for reuse and consistency in development
 - The [Docker image used in this course](#) includes the PULP-SDK and its related compilation toolchain



03. Prerequisites – Docker Setup (WSL install)



- **Step 0** (for Windows users only!) – install Windows Subsystem for Linux (WSL 2)
 - **What is WSL?**
 - **A compatibility layer** that lets you run a genuine Linux environment directly on Windows, without a separate virtual machine or dual boot
 - Gives access to Linux command-line tools and applications, running natively alongside your Windows app
 - It can run both command-line and graphical Linux apps directly inside Windows
 - The **advantage** of running Docker through WSL is that, while in both cases Docker runs through a virtual machine, the WSL is much lighter and it doesn't slow-down the entire system
 - If you plan to work on the lab on a Windows machine, please [follow these installation instructions for WSL](#)

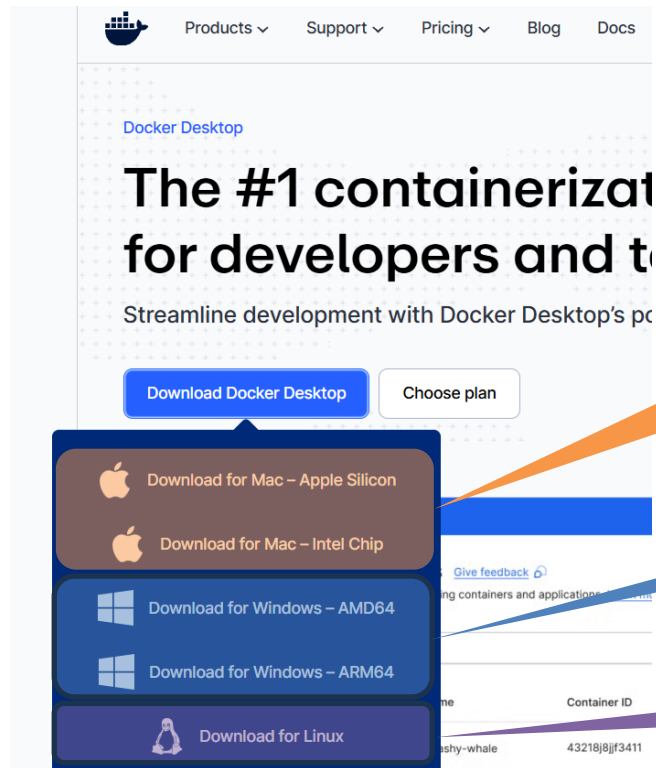


03. Prerequisites – Docker Setup



- Step 1 – install Docker Desktop

- Choose, download and install the proper version from the [official website](#).



- Apple Mac versions
- Choose based on your computer's processor

- Windows versions (AMD64 for most, unless you are **VERY SURE** that your processor is ARM64)
- **CHOOSE THIS EVEN IF YOU WORK WITH WSL!**

- Universal Linux version
- **If working with WSL, choose a Windows version!**



03. Prerequisites – VS Code



- Step 2 – Install Virtual Studio (VS) Code

- What is VS Code?

- A free, popular and versatile text editor from Microsoft
- It is highly customizable through a large extension library
- The one that interests us is the **Dev Containers** extension, that enables VS Code to work with Containers

- How to install?

- Choose, download and install for your operating system from the [official website](https://code.visualstudio.com/)
- **Once VS Code installed**, we need to install the extension for working with Containers
 - Find and install it from the [VS Code Marketplace](https://marketplace.visualstudio.com/items?itemName=ms-vscode-remote-container-devcontainer)

1. Click on the *Install* button

2. In the pop-up, confirm that you want to open VS Code

3. Click on the *Install* button



03. Prerequisites – git

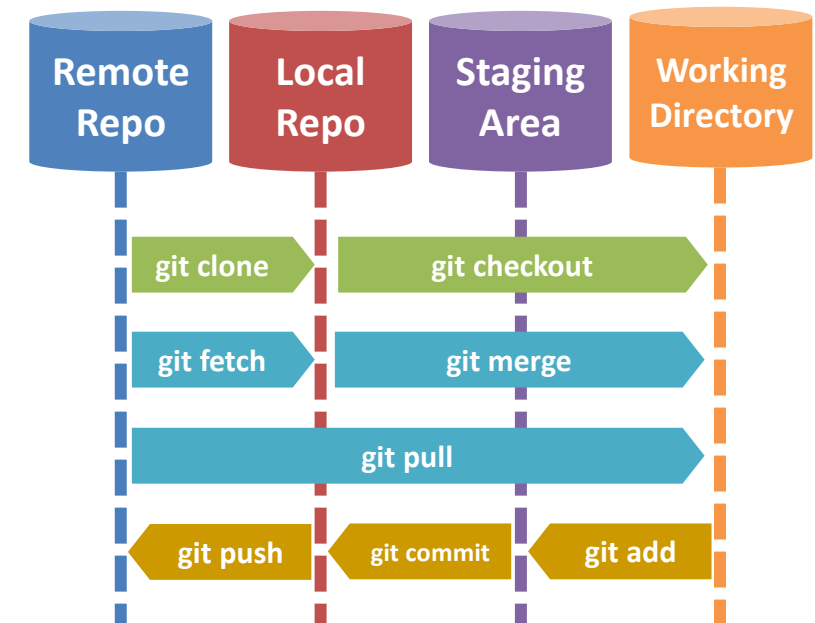
[Link to a useful intro git tutorial](#)

- What is Git?

- A free, open-source version control system that tracks changes to files over time.
- It lets multiple people work on the same codebase (repository) and merge changes (commits) without overwriting each other's work.
- It keeps a full history of changes, so you can review, compare, or revert to any earlier version.

- Common commands we will use

- **git clone** – Downloads a full copy of a remote repository to your machine
- **git checkout** – Switches your working directory to a different branch or commit
- **git fetch** – Checks the remote for new changes without applying them locally
- **git merge** – Combines changes from one branch into another
- **git pull** – Fetches (*downloads*) remote changes and merges them into your local branch (equivalent to git fetch + git merge)
- **git add [file1, file2, ...]** – Stages (*marks*) the specified files to be included in the next commit
- **git commit -m “commit message”** – Saves the staged changes to your local repository's history with a message
- **git push** – Uploads your committed changes to the remote repository



03. Prerequisites – Hello, World Example – Part 1

In order to confirm that everything was installed correctly, we will now run the ***Hello, World*** basic example available in the repository of this lab.

The **example is run on WSL, on a Windows** machine, but it should work identically on other operating systems.

- **Step 1:** In a terminal, open the location where you wish to work

```
calind@LenovoCalin: ~/APAI_LAI  ×  +  v
(base) calind@LenovoCalin:~$ cd APAI_LAB/
(base) calind@LenovoCalin:~/APAI_LAB$
```

- **Step 2:** Clone the repository of lab 1 locally

```
git clone https://github.com/EEESlab/APAI26-LAB01-PULP-Embedded-Programming.git
```

- **Step 3:** Enter the cloned directory

```
cd APAI26-LAB01-PULP-Embedded-Programming/
```

- **Step 4:** Open the directory in VS Code

```
code .
```

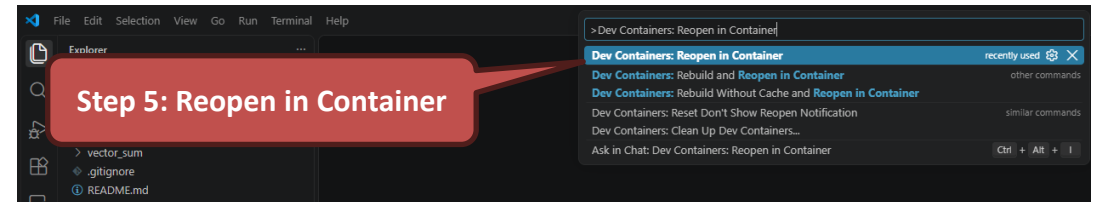


03. Prerequisites – Hello, World Example – Part 2

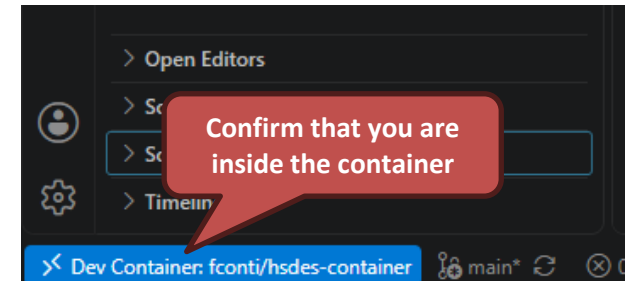
* From this point on, we will only work from VS Code, so it is ok to close the terminal you have previously opened.

- **Step 5:** At this point, the project is open in the local *environment*. We need to **reopen inside the Docker container**.

- **Step 5.1:** Press **CTRL + Shift + P**
- **Step 5.2:** Type in **“Dev Containers: Reopen in Container”**
- **Step 5.3:** Press **Enter**

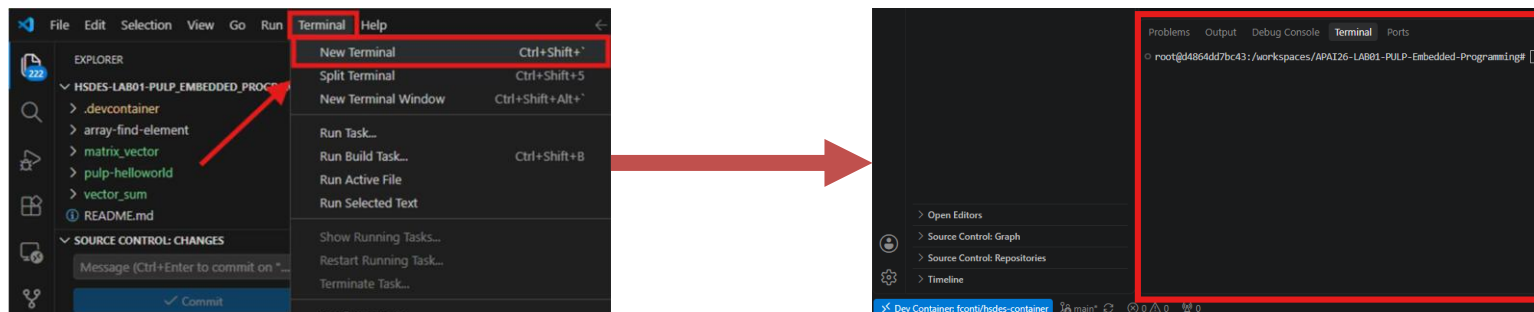


* We are now **inside the container**. From now on, when opening VS Code, the project will automatically open inside the Docker Container. It is confirmed in the bottom-left corner of VS Code.



- **Step 6:** Open a **terminal inside VS Code** (which will be **inside the container**)

- Access the terminal through the **top-left task bar (Terminal -> New terminal)**
- The terminal **will open in the bottom** of the window.

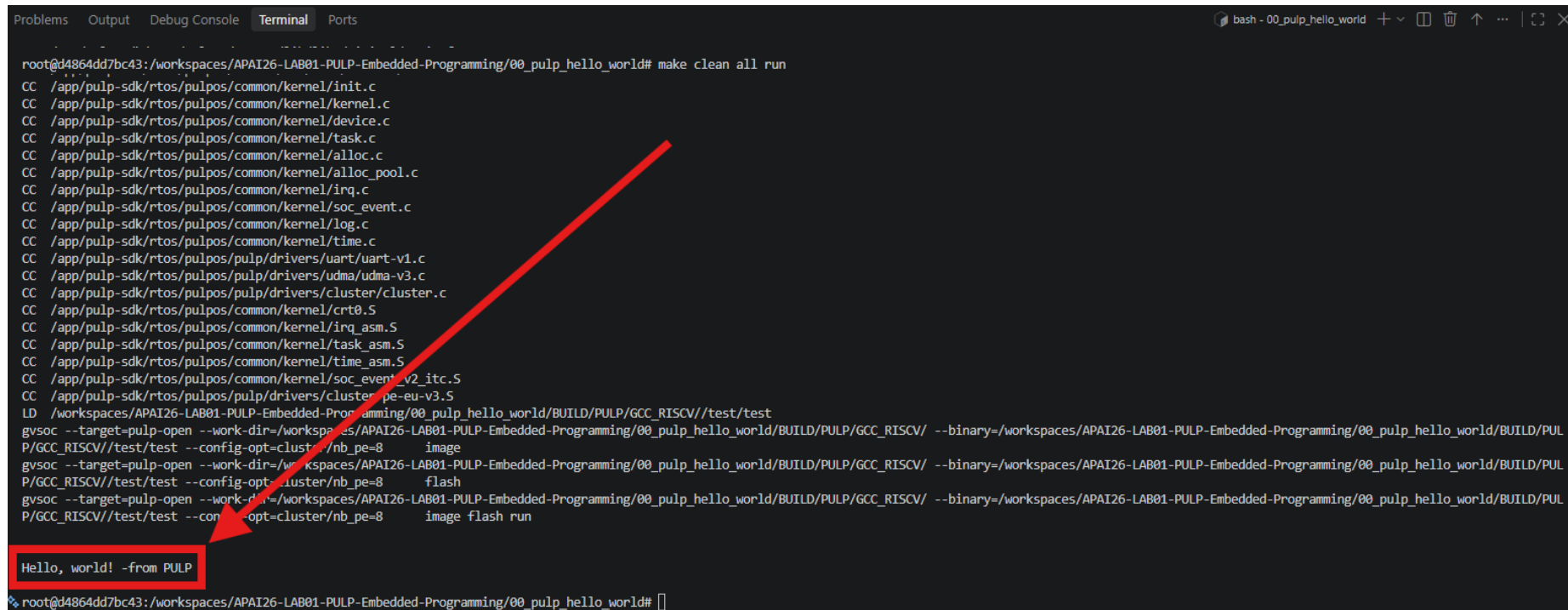


03. Prerequisites – Hello, World Example – Part 3

- **Step 7:** Run the *Hello, World* example

```
cd 00_pulp_hello_world/  
make clean all run
```

- If everything went well, you should now see the “*Hello, world! -from PULP*” message in the VS Code terminal.



```
root@d4864dd7bc43:/workspaces/APAI26-LAB01-PULP-Embedded-Programming/00_pulp_hello_world# make clean all run  
CC /app/pulp-sdk/rtos/pulpos/common/kernel/init.c  
CC /app/pulp-sdk/rtos/pulpos/common/kernel/kernel.c  
CC /app/pulp-sdk/rtos/pulpos/common/kernel/device.c  
CC /app/pulp-sdk/rtos/pulpos/common/kernel/task.c  
CC /app/pulp-sdk/rtos/pulpos/common/kernel/alloc.c  
CC /app/pulp-sdk/rtos/pulpos/common/kernel/alloc_pool.c  
CC /app/pulp-sdk/rtos/pulpos/common/kernel/irq.c  
CC /app/pulp-sdk/rtos/pulpos/common/kernel/soc_event.c  
CC /app/pulp-sdk/rtos/pulpos/common/kernel/log.c  
CC /app/pulp-sdk/rtos/pulpos/common/kernel/time.c  
CC /app/pulp-sdk/rtos/pulpos/pulp/drivers/uart/uart-v1.c  
CC /app/pulp-sdk/rtos/pulpos/pulp/drivers/udma/udma-v3.c  
CC /app/pulp-sdk/rtos/pulpos/pulp/drivers/cluster/cluster.c  
CC /app/pulp-sdk/rtos/pulpos/common/kernel/crt0.S  
CC /app/pulp-sdk/rtos/pulpos/common/kernel/irq_asm.S  
CC /app/pulp-sdk/rtos/pulpos/common/kernel/task_asm.S  
CC /app/pulp-sdk/rtos/pulpos/common/kernel/time_asm.S  
CC /app/pulp-sdk/rtos/pulpos/common/kernel/soc_event_v2_itc.S  
CC /app/pulp-sdk/rtos/pulpos/pulp/drivers/cluster/pe-eu-v3.S  
LD /workspaces/APAI26-LAB01-PULP-Embedded-Programming/00_pulp_hello_world/BUILD/PULP/GCC_RISCV/test/test  
gvsoc --target=pulp-open --work-dir=/workspaces/APAI26-LAB01-PULP-Embedded-Programming/00_pulp_hello_world/BUILD/PULP/GCC_RISCV/ --binary=/workspaces/APAI26-LAB01-PULP-Embedded-Programming/00_pulp_hello_world/BUILD/PULP/GCC_RISCV/test/test --config-opt=cluster/nb_pe=8 image  
gvsoc --target=pulp-open --work-dir=/workspaces/APAI26-LAB01-PULP-Embedded-Programming/00_pulp_hello_world/BUILD/PULP/GCC_RISCV/ --binary=/workspaces/APAI26-LAB01-PULP-Embedded-Programming/00_pulp_hello_world/BUILD/PULP/GCC_RISCV/test/test --config-opt=cluster/nb_pe=8 flash  
gvsoc --target=pulp-open --work-dir=/workspaces/APAI26-LAB01-PULP-Embedded-Programming/00_pulp_hello_world/BUILD/PULP/GCC_RISCV/ --binary=/workspaces/APAI26-LAB01-PULP-Embedded-Programming/00_pulp_hello_world/BUILD/PULP/GCC_RISCV/test/test --config-opt=cluster/nb_pe=8 image flash run  
  
Hello, world! -from PULP  
root@d4864dd7bc43:/workspaces/APAI26-LAB01-PULP-Embedded-Programming/00_pulp_hello_world#
```



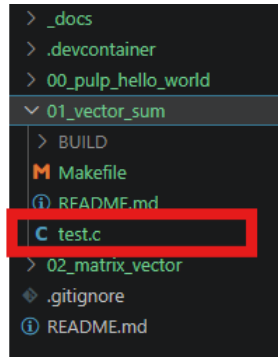


ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

04. Assignments

04. Assignments – Task 01: Vector Sum – test.c Breakdown

- Let's open the main file (**test.c**) of the vector sum exercise and analyze it



```
#include "pmsis.h"

#define N 50 // array size
unsigned char array_1[N]; // define array of size N. [i]

// initialize an array A_ar with _size_ number of elements
// and precompute the result (we use this as a checksum)
int init_array(unsigned char * A_ar, int size) { ...
}

// print array elements
void print_array(unsigned char * A_ar, int size) { ...
}

// return the index of the item==element of the array A_ar
int vector_sum(unsigned char * A_ar, int size) { ...
}

int main() { ...
}
```

Define the **array size** (defaults to 50)

init_array() – initialize a given array with the element having the value equal to its index + 1, and returns its sum according to the mathematical definition $(N * (N - 1)) / 2$

print_array() – iterates through the elements of the given array and prints each of them

vector_sum() – it is the function doing the actual computation, summing all the elements

```
int main() {
    printf("\n\nThis example program computes the sum of %d integer numbers!\n", N);

    // initialize the array and get the expected result
    int expected_result = init_array(array_1, N);

    // inspect the array. let's print its values !
    print_array(array_1, N);

    // call the vector_sum function
    int computed_result = vector_sum(array_1, N);

    // check if the results is correct
    if (computed_result == expected_result)
        printf("Result is correct. Got %d\n\n", computed_result);
    else
        printf("Result is not correct. Got %d instead of %d\n\n", computed_result, expected_result);
}
```

The main function calls all the functions previously defined to perform the 3 steps: initialization, print, and sum

Finally, we check against the mathematical reference given by the init function

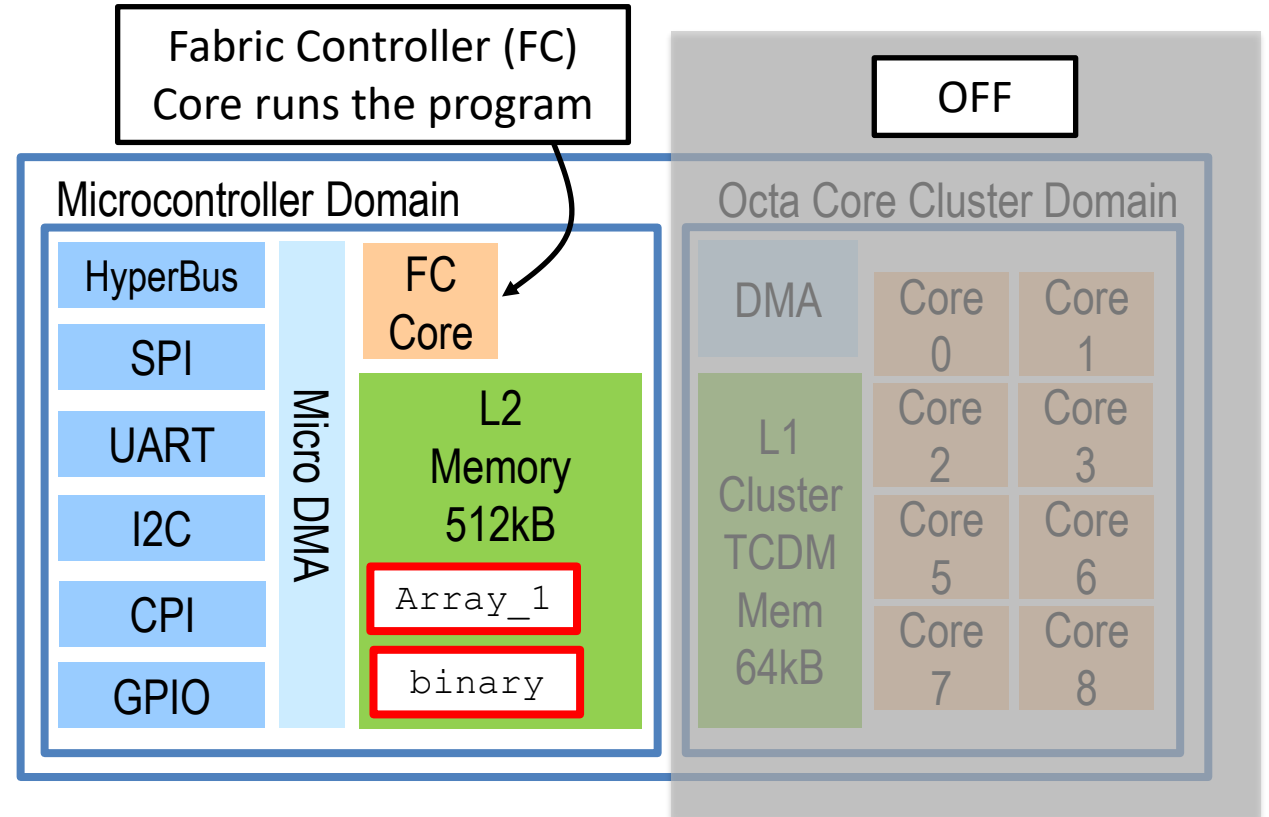


04. Assignments – Task 01: Vector Sum – Setup

- The exercise will be **run on the Fabric Controller (FC)** – no need to worry about cluster parallelism for now
- **Exercise setup**
 - In the terminal, be sure you are inside the “01_vector_sum” directory
 - If you are still in the “00_pulp_hello_world” directory, from the earlier steps, run the command below, which will take you in the parent directory (*cd = current directory*)

```
cd ..
```

```
cd 01_vector_sum
```



04. Assignments – Task 01: Vector Sum – Exercises

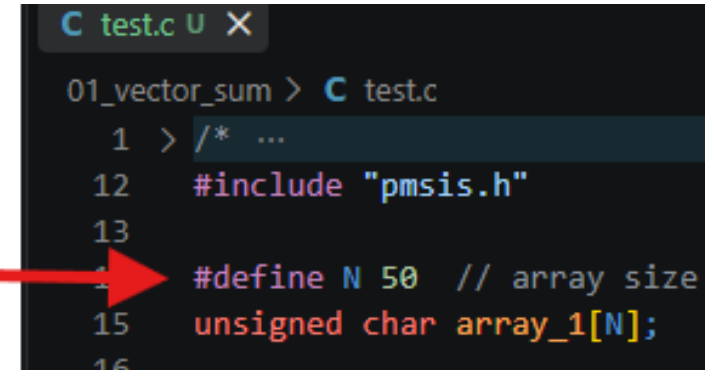
Exercise 1.1

- Be sure that N (the vector size) is set to **50** in test.c
- **Run the code** with make

```
make clean all run
```

- Fill in the **assignment template**
- **Tips:**

- **Question 3:** to print the element of the array, uncomment the function call “*print_array(array_1, N)*” inside the **main()**



```
C test.c U X
01_vector_sum > C test.c
1 > /* ...
12 #include "pmsis.h"
13
14 #define N 50 // array size
15 unsigned char array_1[N];
16
```

Exercise 1.2

- Define **N to 350** and then **run the code** and **fill in the assignment**, like in 1.1
- **Tips:**
 - **Question 4:** what **range of values** a “char” data type can represent? Check the **comment in the code**, on **line 15**

Exercise 1.3

- Fix the issue from **Exercise 1.2**
- Cast “array_1” and the function’s arguments to **int (or short int)**, rerun with make, and **fill in the template**



04. Assignments – Task 01: Vector Sum – Explanation for 1.3

- When you run **Exercise 1.2**, you should get the output below

```
This example program computes the sum of 350 integer numbers!  
Result is not correct. Got 37105 instead of 61425
```

- Looking at the initializations in the **main file**, we can notice that the **array_1** is of type **unsigned char**

```
#define N 350 // array size  
unsigned char array_1[N];
```

- **unsigned char** is represented using only **8 bits**, so the **range** of values it can take is **between 0 and $2^8 - 1 = 255$**
- Thus, for every element above 255, the assigned **value will get truncated to $value \% 256$**
- If you will print the array values, you can see this pattern happening:

```
1 2 3 4 5 6 7 8 9 10 11 12 13 14 [...] 247 248 249 250 251 252 253 254 255 0 1 2 3 4 5 6 7 8 [...] 88 89 90 91 92 93 94 [...]
```

- The solution is to define the array as **int** or **unsigned int** instead (represented on **16 bits**, so with possible values up to **32,767** for signed, and **65,535** for unsigned. **Remember to make this change in all the function arguments as well!**

```
15 unsigned int array_1[N];
```

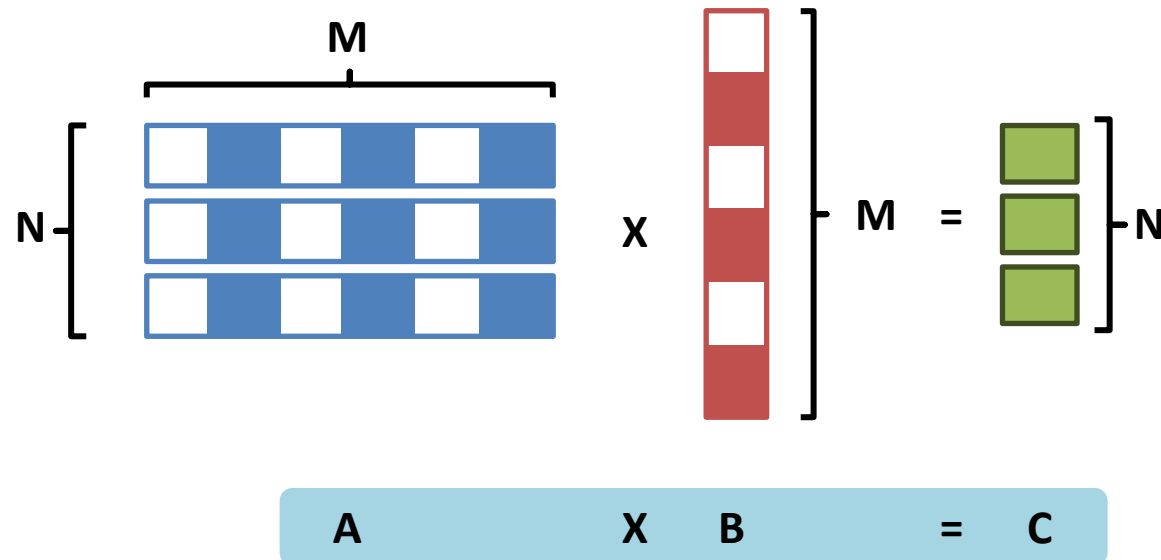
```
19 int init_array(unsigned int * A_ar, int size) {
```

```
27 void print_array(unsigned int * A_ar, int size) {
```

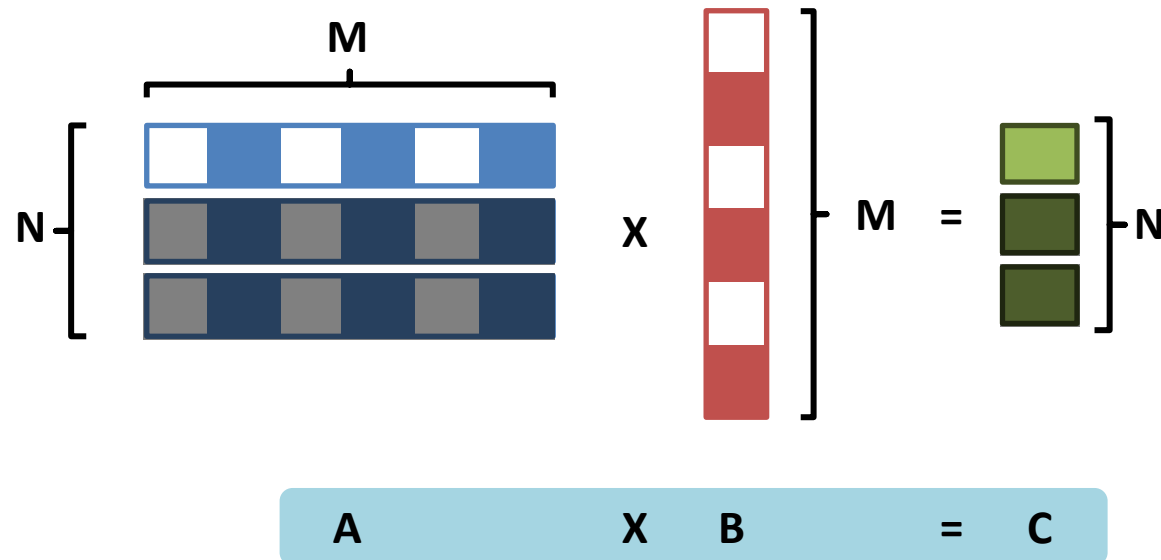
```
35 int vector_sum(unsigned int * A_ar, int size) {
```



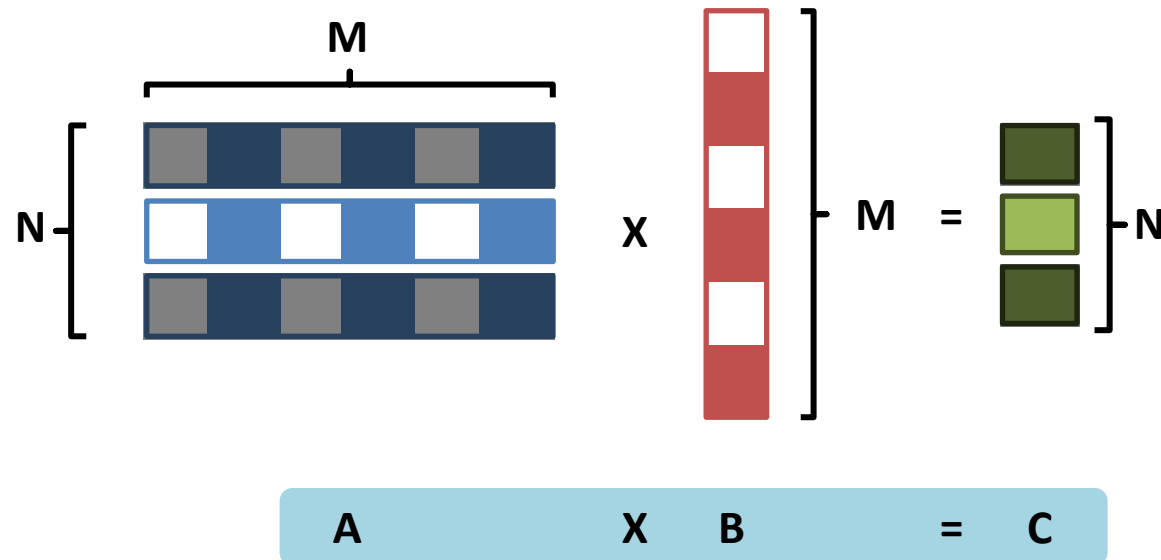
04. Assignments – Task 02: Matrix-Vector Product – Intro



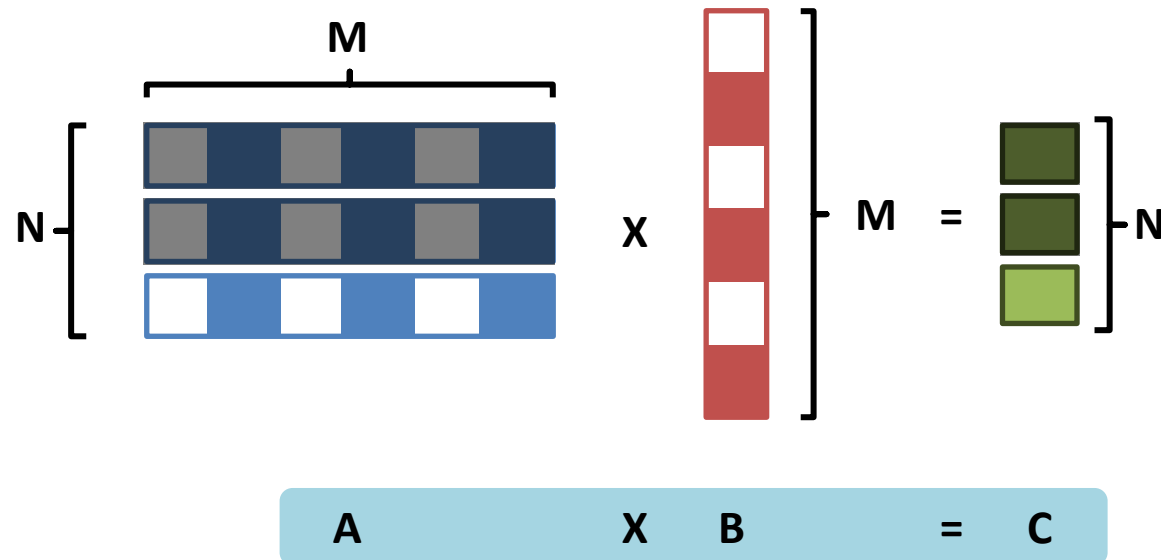
04. Assignments – Task 02: Matrix-Vector Product – Intro



04. Assignments – Task 02: Matrix-Vector Product – Intro

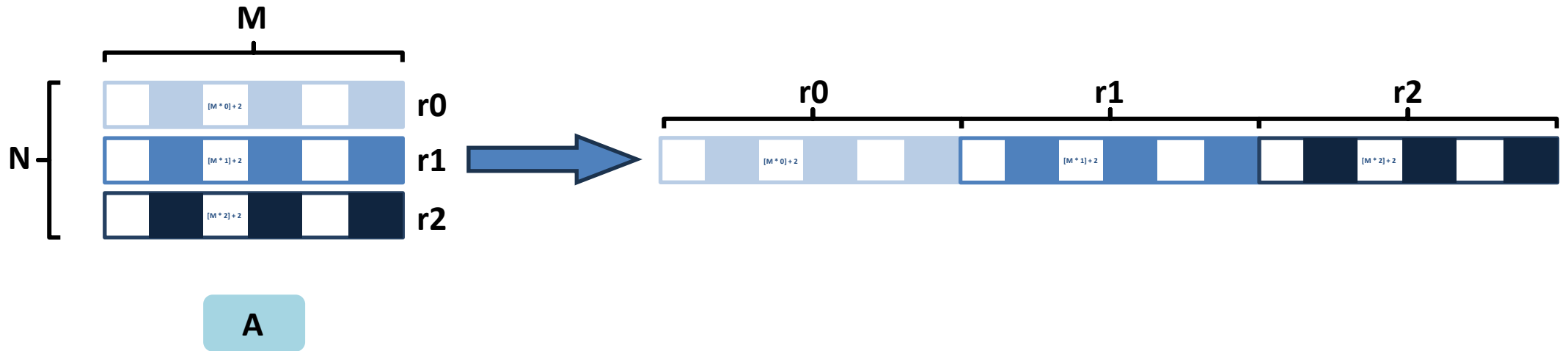


04. Assignments – Task 02: Matrix-Vector Product – Intro

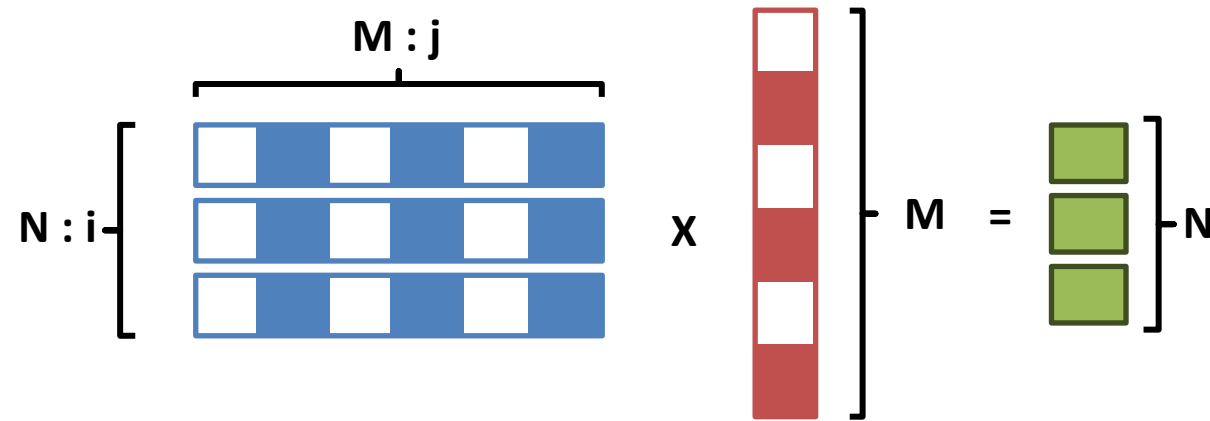


04. Assignments – Task 02: Matrix-Vector Product – Intro

- A **multi-dimensional array**, including a **matrix**, is efficiently represented as a **memory-contiguous array**
- In general: $\text{Mat}[i][j] \rightarrow M[i * M + j]$



04. Assignments – Task 02: Matrix-Vector Product – Intro



mat $\times$ vec = out_v

```
// generic matrix-vector multiplication
int gemv(int M, int N, float *mat, float *vec, float *out_v){

    for (int i=0; i<N; i++){
        out_v[i] = 0;

        for (int j=0; j<M; j++){
            out_v[i] += mat[i * M + j] * vec[j];
        }
    }
}
```


04. Assignments – Task 02: Matrix-Vector Product – Assembly Code

- A **disassembler** is a computer program that **translates machine language into assembly language** - the **inverse operation to that of an assembler**.
- To **produce the assembly code** and save it in a *dis.txt* file, use the command:

```
make dis > dis.txt
```

04. Assignments – Task 02: Matrix-Vector Product – Assembly Code

```
1c008706 <gemv>:
1c008706: 02a05d63      blez a0,1c008740 <gemv+0x3a>
1c00870a: 00259f13      slli t5,a1,0x2
1c00870e: 8fb6         mv t6,a3
1c008710: 01e68e33      add t3,a3,t5
1c008714: 4e81         li t4,0
1c008716: a005         j 1c008736 <gemv+0x30>
1c008718: 0048230b      p.lw t1,4(a6!)
1c00871c: 0046a88b      p.lw a7,4(a3!)
1c008720: 431c         lw a5,0(a4)
1c008722: 431307b3      p.mac a5,t1,a7
1c008726: c31c         sw a5,0(a4)
1c008728: ffc698e3      bne a3,t3,1c008718 <gemv+0x12>
1c00872c: 0e85         addi t4,t4,1
1c00872e: 967a         add a2,a2,t5
1c008730: 0711         addi a4,a4,4
1c008732: 01d50763      beq a0,t4,1c008740 <gemv+0x3a>
1c008736: 86fe         mv a3,t6
1c008738: 8832         mv a6,a2
1c00873a: fcb04fe3      bgtz a1,1c008718 <gemv+0x12>
1c00873e: b7fd         j 1c00872c <gemv+0x26>
1c008740: 8082         ret
```

```
// generic matrix-vector multiplication
void gemv(int size_N, int size_M,
          float * mat_I, float *vec_i, float * vec_o)
{
    for (int i=0; i<size_N; i++){
        for (int j=0; j<size_M; j++){
            // mulitply accumulate operation
            vec_o[i] += mat_i[i*size_M+j] * vec_i[j];
        }
    }
}
```

Inner
loop

In the **Makefile**:

1. Change from -O3 to -O1. What has changed in the assembly?
2. Remove -mnohwloop. What has changed in the assembly?



04. Assignments – Task 02: Matrix-Vector Product – Assembly Code

Makefile

APP = matrix-vector

APP_SRCS += test.c

APP_CFLAGS += -O3 -g -mnohwloops

APP_LDFLAGS +=

include \$(RULES_DIR)/pmsis_rules.mk

```
1c008706 <gemv>:
1c008706: 02a05c63      blez a0,1c00873e <gemv+0x38>
1c00870a: 02b05a63      blez a1,1c00873e <gemv+0x38>
1c00870e: 00259e93      slli t4,a1,0x2
1c008712: 050a         slli a0,a0,0x2
1c008714: 00a70e33      add t3,a4,a0
1c008718: 01d68333      add t1,a3,t4
1c00871c: 0047258b      p.lw a1,4(a4!) # 10004 <__l1_heap_size+0x20>
1c008720: 87b6         mv a5,a3
1c008722: 8532         mv a0,a2
1c008724: 0045288b      p.lw a7,4(a0!)
1c008728: 0047a80b      p.lw a6,4(a5!)
1c00872c: 430885b3      p.mac a1,a7,a6
1c008730: feb72e23      sw a1,-4(a4)
1c008734: fef318e3      bne t1,a5,1c008724 <gemv+0x1e>
1c008738: 9676         add a2,a2,t4
1c00873a: feee11e3      bne t3,a4,1c00871c <gemv+0x16>
1c00873e: 8082         ret
```

The -O3 optimized code gets rid of a `/lw` instruction because the accumulator is kept in the register file!

```
// generic matrix-vector multiplication
void gemv(int size_N, int size_M,
float * mat_I, float *vec_i, float * vec_o)
{
    for (int i=0; i<size_N; i++){
        for (int j=0; j<size_M; j++){
            // mulitply accumulate operation
            vec_o[i] += mat_i[i*size_M+j] * vec_i[j];
        }
    }
}
```

Inner
loop



04. Assignments – Task 02: Matrix-Vector Product – Assembly Code – HW Loops

Makefile

APP = matrix-vector

APP_SRCS += test.c

APP_CFLAGS += -O3 -g ~~-mnohwloops~~

APP_LDFLAGS +=

include \$(RULES_DIR)/pmsis_rules.mk

REMOVE "NO HW LOOPS" FLAG

1c008706 <gemv>:

```
1c008706: 04a05463    blez      a0,1c00874e <ge
1c00870a: 04b05263    blez      a1,1c00874e <ge
1c00870e: 00251e13    slli      t3,a0,0x2
1c008712: 1e71        addi      t3,t3,-4
1c008714: 00259e93    slli      t4,a1,0x2
1c008718: 002e5e13    srli      t3,t3,0x2
1c00871c: 01d68f33    add       t5,a3,t4
1c008720: 0e05        addi      t3,t3,1
1c008722: 015e407b    lp.setup  x0,t3,1c00874c <gemv+0x46>
1c008726: 40df07b3    sub       a5,t5,a3
1c00872a: 17f1        addi      a5,a5,-4
1c00872c: 0047258b    p.lw      a1,4(a4!) # 10004 <__l1_heap_size+0x20>
1c008730: 8389        srli      a5,a5,0x2
1c008732: 8836        mv        a6,a3
1c008734: 8532        mv        a0,a2
1c008736: 0785        addi      a5,a5,1
1c008738: 0087c0fb    lp.setup  x1,a5,1c008748 <gemv+0x42>
1c00873c: 0045230b    p.lw      t1,4(a0!)
1c008740: 0048288b    p.lw      a7,4(a6!)
1c008744: 431305b3    p.mac     a1,t1,a7
1c008748: feb72e23    sw        a1,-4(a4)
1c00874c: 9676        add       a2,a2,t4
1c00874e: 8082        ret
```

Hardware Loops Instructions have been placed!

```
// generic matrix-vector multiplication
void gemv(int size_N, int size_M,
float * mat_I, float *vec_i, float * vec_o)
{
    for (int i=0; i<size_N; i++){
        for (int j=0; j<size_M; j++){
            // mulitply accumulate operation
            vec_o[i] += mat_i[i*size_M+j] * vec_i[j];
        }
    }
}
```

Inner
loop



04. Assignments – Task 02: Matrix-Vector Product – Exercises

Before starting: be sure you work from the directory of this exercise! Run the usual commands

```
cd ..  
cd 02_matrix_vector
```

Exercise 2.1

- **Goal:** *manually* count **the number of multiply-accumulate (MAC)** operations performed for a GEMV
- **How:** working in the *int gemv()* function, **increment a counter** every time a MAC is performed inside the GEMV inner loop. For this purpose, use the available variable *mac_counter*
- **Hint:** look for **comments** that are meant to guide you throughout the codebase
- **Paste** the resulting code in the **assignment template**

```
// multiply accumulate operation (MAC)  
vec_o[i] += mat_i[i * size_M + j] * vec_i[j];
```

Exercise 2.2

- **Goal:** add **performance counters** to profile the GEMV
- **How:** Use the *start_perf_counter();* and *stop_perf_counter();* functions that are available. The profiling must start immediately before and finish immediately after the function of interest!
- **Hint:** work inside the **main function**, including just the call of the *gemv()* function when profiling
- **Paste** the resulting code in the **assignment template**

```
start_perf_counter();  
stop_perf_counter();
```



04. Assignments – Task 02: Matrix-Vector Product – Exercises

Exercise 2.3

- **Goal:** Enable the **performance counters** of interest. You need to profile
 - Execution cycles (total)
 - Number of instructions executed
- **How:** Work inside the *start_perf_counter()* function, in *02_matrix_vector/test.c* and fill in the code in the marked parts
- **Hint:** use the **list in the right** side to find the correct perf event names. They can also be found in:

`/app/pulp-sdk/rtos/pmsis/pmsis_api/include/pmsis/chips/default.h`

- **Paste** the resulting code in the **assignment template**

```
typedef enum {
    PI_PERF_CYCLES      = 17, /*!< Total number of cycles (also includes the
                                cycles where the core is sleeping). Be careful that this event is using a
                                timer shared within the cluster, so resetting, starting or stopping it on
                                one core will impact other cores of the same cluster. */
    PI_PERF_ACTIVE_CYCLES = 0, /*!< Counts the number of cycles the core was
                                active (not sleeping). */
    PI_PERF_INSTR        = 1, /*!< Counts the number of instructions executed.
                                */
    PI_PERF_LD_STALL     = 2, /*!< Number of load data hazards. */
    PI_PERF_JR_STALL     = 3, /*!< Number of jump register data hazards. */
    PI_PERF_IMISS        = 4, /*!< Cycles waiting for instruction fetches, i.e.
                                number of instructions wasted due to non-ideal caching. */
    PI_PERF_LD           = 5, /*!< Number of data memory loads executed.
                                Misaligned accesses are counted twice. */
    PI_PERF_ST           = 6, /*!< Number of data memory stores executed.
                                Misaligned accesses are counted twice. */
    PI_PERF_JUMP         = 7, /*!< Number of unconditional jumps (j, jal, jr,
                                jalr). */
    PI_PERF_BRANCH       = 8, /*!< Number of branches. Counts both taken and
                                not taken branches. */
    PI_PERF_BTAKEN       = 9, /*!< Number of taken branches. */
    PI_PERF_RVC          = 10, /*!< Number of compressed instructions
                                executed. */
    PI_PERF_LD_EXT       = 12, /*!< Number of memory loads to EXT executed.
                                Misaligned accesses are counted twice. Every non-TCDM access is considered
                                external (cluster only). */
    PI_PERF_ST_EXT       = 13, /*!< Number of memory stores to EXT executed.
                                Misaligned accesses are counted twice. Every non-TCDM access is considered
                                external (cluster only). */
    PI_PERF_LD_EXT_CYC   = 14, /*!< Cycles used for memory loads to EXT.
                                Every non-TCDM access is considered external (cluster only). */
    PI_PERF_ST_EXT_CYC   = 15, /*!< Cycles used for memory stores to EXT.
                                Every non-TCDM access is considered external (cluster only). */
    PI_PERF_TCDM_CONT    = 16, /*!< Cycles wasted due to TCDM/log-interconnect
                                contention (cluster only). */
} pi_perf_event_e;
```



04. Assignments – Task 02: Matrix-Vector Product – Exercises

Exercise 2.4

- **Goal:** use the quantities we obtain from the **profiling** we enabled so far to get more insight into the **performance** of our program
- **How:** in *test.c*, in the *stop_perf_counter()* function, fill in the */*YOUR CODE HERE*/* parts, to obtain:
 - CPI = cycles/instructions
 - MAC/cycles
 - Instructions/cycles
 - Instructions/MACs
- **Paste** the resulting code in the **assignment template**

Exercise 2.5

- **Goal:** use everything we've done so far to **profile the GEMV function**, for a size of $M = N = 50$
- **How:** run the program using *make*, **profile the matrix-vector product** with different compiler optimizations: **O1, O3, and O3 with hardware loops**
- **Fill in the table in the assignment template**

```
make clean all run
```





ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Călin Diaconu

For questions, contact us at:

`calin.diaconu2@unibo.it`

www.unibo.it